INTERNAL · QA AUTOMATION

QA Automation — access, architecture, pipeline and coverage

FIELD	DETAIL
Prepared by	QA Analyst — Automation
Date	1 September 2026
Audience	Executive sponsor, QA lead, engineering
Supersedes	The QA Automation brief circulated earlier on 1 September 2026
Supporting detail	Repository documentation, listed in the appendix
Decisions requested	Five, in section 9

Summary

The automation framework exists as working code in the organisation's repository, it has been run against the live storefront, and — contrary to the previous version of this brief — **continuous integration is already enabled and passing.** Thirty consecutive green runs since 26 August gate every push: types, lint, review, 324 unit tests and all nine detection controls, in about four minutes.

What has never happened is a single scheduled run against the store. The nightly job has been skipped thirty times out of thirty, and the reason is narrow and fixable: `main` **is the default branch and contains no workflow files.** GitHub fires `schedule` triggers only from the default branch, so the nightly, release and report tiers cannot start, and four of the five committed workflows are not registered at all.

That is a branch operation, not an engineering project. It replaces "choose and build a pipeline" as the first decision in this brief.

The live runs produced real findings — untranslated navigation in all four supported locales, localized product pages served in English, and a welcome-kit buy button that adds nothing to the cart. They also produced roughly four times as many failures that were about the harness rather than the store, and the larger part of recent work has been making that difference legible. A report that cannot separate "the store is broken" from "we could not look" is a report that gets ignored, and then switched off.

The tooling costs between zero and forty dollars a month. The constraint has never been budget.

1. Access to the automation

AUDIENCE	ROUTE	STATUS
Engineering and QA	Organisation repository. Clone, install dependencies, run `npm run verify`	Live, via organisation membership
Anyone reviewing results	Allure report generated on whichever machine ran the suite	Gap — no shared link, no digest
Sponsors and stakeholders	No route today	Proposed: daily digest plus a published report link

Read access to the repository is not equivalent to access to the results. Cloning a repository and installing browsers is a reasonable requirement for an engineer and an unreasonable one for anybody else. The pipeline is what produces a link and a digest.

2. Architecture documentation

Documented in the repository and current. No approval required; recorded because it was previously listed as outstanding.

DOCUMENT	CONTENTS
`README`	Entry point. Layers, data flow, the standing rules, configuration reference
`docs/LIVE-RUN.md`	What the live runs found, what was harness, and what each remaining gap needs
`docs/TEST-CASE-COVERAGE.md`, `docs/TRACEABILITY.md`	Every risk area, its coverage, and what is knowingly uncovered
`docs/ADDING-A-CHECK.md`	How a new check is added, including its mandatory detection control
Per-area docs	`ACCESSIBILITY`, `VISUAL-REGRESSION`, `TRANSLATION-BACKLOG`, `TOP-PRODUCTS-DAILY`, `AD-LANDING-DAILY`, `COMPARE-AT-AUDIT`, `WELCOME-KIT-COVERAGE`, `REPORTING`

3. What is built

3.1 Translation — the largest single body of work

Translation is where most of the engineering has gone, and it is the area closest to replacing a recurring manual pass outright.

COMPONENT	STATE	WHAT IT DOES
Content-layer parity engine	Built. 60 unit tests	Compares every translatable string across locales without a browser. Severities, exemptions, protected terms and thresholds all declared in `config/i18n.yaml`, never compiled in
Shopify translations collector	**Built**, never run	Read-only Admin GraphQL `translatableResources`. Paginates, and distinguishes "fetch failed" from "value absent" — collapsing those two turns an outage into a green run. Needs a token; see §8
Render-layer locale suite	Built. 244 generated test cases	What the DOM shows that the API cannot: locale routing, `html[lang]`, hreflang, price format, meta tags, encoding integrity, layout overflow at 390px
Translation backlog audit	Built. 29 unit tests	Takes the 93 open "Translate Product" tasks and asks the live storefront whether each is still true
Asana pull and close	Built	`asana:pull` reads the board; `asana:close` closes only the tasks the audit proved done. Dry-run by default. Refuses partial, unverified and stale-product tasks
Detection control	Built	13 planted defects, 13 caught. 59 render specs fail against a knowingly broken storefront, as they must

The locale contract is four markets plus English: ES, DE, IT, FR. The original test plan asked for seven. Korean and Japanese were removed rather than skipped, because the store does not serve them — checking them produced sixty-odd failures a run about routes that 404 and scripts that are absent because there is no copy in them. They are recorded as **unverified, not passing**, and the patterns are preserved in version history if either market goes live.

3.2 Everything else built

AREA	STATE
Unit tests	324 across 16 files
Daily audits	Ad-traffic landing pages, top-10 sellers, compare-at removal, translation backlog, accessibility by market, locale render specs — one command, one report
Accessibility	**Built** — WCAG 2.2 AA per market, 21 unit tests, detection control. The previous brief deferred this
Visual regression	**Built** — 10 committed baselines, masks that each require a written reason, separate fixture and live baseline trees, detection control. The previous brief deferred this
Welcome-kit parity	Built. Compares kits in the cart, with a `canonical_title` identity backstop that has already caught two renamed products
Health-check re-verification	Built. Re-asks every issue in a QA report daily; reports "not reproduced" rather than "invalid"
Release gate	Built. 22 stages across an offline tier and a live tier; the offline tier must be green before a live result counts as evidence
Detection controls	9 of them. Every check has been watched to fail on a known-broken fixture
Guardrails	Custom lint plugin blocking production writes and floating-point money; pre-commit hooks; offline reviewer

AREA	STATE
Fixtures	8 deterministic local servers, so the whole suite runs offline with no store and no credentials
Mobile	Maestro smoke flow, launch to checkout
Pipeline	5 committed workflows. One registered and passing on every push; the scheduled tiers have never fired — see §4

4. Pipeline: what runs today, and the one thing stopping the rest

The previous brief said there is no pipeline. That is not accurate. There is one, it is enabled, and it has been green for a week — but only the half of it that never touches the store.

What runs on every push

JOB	CONTENTS	RESULT
`guardrails`	TypeScript, ESLint and the custom Kitsch rules, offline reviewer, 324 unit tests	Passing, ~40 seconds
`controls`	All nine detection controls — every suite proven to fail against a knowingly broken fixture	Passing, ~3.5 minutes

Thirty runs since 26 August, every one successful. The harness itself is genuinely gated.

What has never run

JOB OR WORKFLOW	TRIGGER	RUNS TO DATE
`pipeline.yml` → `nightly`	03:00 UTC schedule	**0** — skipped 30 of 30
`pipeline.yml` → `release`, `nightly-report`	Tags, schedule	**0**
`translation-gate.yml`	09:30 UTC nightly, PRs touching i18n	**0** — not registered
`web-matrix.yml`	09:00 UTC weekdays, PRs	**0** — not registered
`daily-top-products.yml`	06:15 UTC daily	**0** — not registered
`daily-ad-landing.yml`	05:45 UTC daily	**0** — not registered

Why

`main` is the default branch and holds no workflow files. It carries documentation only; all five workflows live on `develop`.

GitHub runs `schedule` triggers exclusively from the default branch's latest commit. So no cron in this repository can fire, regardless of what the files say. `pipeline.yml` is registered at all only because pushing it to `develop` registered it, and it runs on `push` — which is why the guardrails half works and the scheduled half is skipped every time. The other four workflows have no `push` trigger, so nothing has ever registered them.

The fix is to merge `develop` into `main`, or repoint the default branch at `develop`. No workflow needs writing or rewriting. Then:

1. Set three secrets: `SHOPIFY_ADMIN_TOKEN` (read-only), `ASANA_TOKEN`, `KITSCH_DEV_STORE_URL`.
2. Name one owner.

`translation-gate.yml` still describes seven locales in its header and `config/a11y.yaml` still lists Japanese. About half a day of drift to clear before the first scheduled run — in §7, because it is a fix rather than a decision.

Why the store-facing half has to leave the laptop

The harness is already gated in CI. Every check that actually looks at mykitsch.com still runs only when someone types a command on a personal Windows machine — which is where all the findings in §5 came from.

- A check that exists on one machine runs when that person remembers, and stops existing when they are on leave or change machines.
- Results from an unmanaged machine cannot be independently reproduced. That is testimony, not evidence.
- Any credential the checks need has to live on that machine. A store token in a `.env` file on a personal computer sits outside device management, rotation and revocation. Holding it as a CI secret resolves the security question and the reproducibility question in the same move.

Platform and cost

The platform question is largely settled by what already works. GitHub Actions is enabled, the workflows are written for it, the code is in a GitHub organisation, and the guardrails tier has a week of green history. Migrating would mean porting five working files to buy nothing.

Kept for completeness, since the suite is only a Node process and any system that can check out a repository and run a test command could host it:

PLATFORM	APPROPRIATE WHEN	COST AT ~4,000 MIN/MONTH	FRICTION
GitHub Actions	**Current state — already enabled and passing**	~$6 on Team; $0 on Enterprise Cloud	**None. Already running**
Azure DevOps	Azure DevOps already runs with an owner and agent capacity	~$40	Porting five workflows, plus a service connection
AWS CodeBuild	Already AWS-native and CI is wanted in the same account and IAM model	~$25	Porting, IAM configuration, an account owner
Jenkins	An instance already exists with a maintainer who is not QA	~$30–35 of host, plus patching	Highest — adds a server to keep running

Migration between them is roughly a day of configuration, so this stays a reversible decision. It is only worth revisiting if another platform is already staffed and the organisation would rather not maintain two.

This is not a deployment pipeline. Deployment is already handled. What is missing is the gate.

Sequencing

OPTION	DELIVERS	REQUIRES
A — scheduled and launch tiers	Nightly drift detection, a launch gate, a published report and a daily digest	Nothing outside this repository. Can begin immediately
B — add the merge gate	Defects stopped before they reach the live site	Access to the storefront repository, a small workflow change there, and deploy-preview URLs reaching CI

Begin with A. Move to B once the flake rate is known. Blocking a merge with a suite whose reliability has not been demonstrated is how teams learn to ignore continuous integration.

5. What the automation found on the live store

These came from the suite, not from a manual pass.

FINDING	LOCALES	STATUS
Navigation labels render in English — account, best sellers, hair, sale, shower, sleep	FR, DE, IT, ES	Confirmed, 20 failures
Localized product pages return 200 and declare `html lang="en"`	FR, DE, IT, ES	Confirmed
Meta titles are not localized on home and product pages	FR, DE, IT, ES	Confirmed, 8 failures
The welcome-kit buy button adds nothing to the cart — `/cart.js` reports 0 items after the press	All	Confirmed. It opens a bundle builder rather than adding a line
Two products in the kit configuration had been renamed on the store	—	Caught by the `canonical_title` backstop

Separately, and worth knowing before the visual suite is trusted: the collection page lost a row of products between two runs three minutes apart. That is merchandising, not a defect, and it is the reason that shot is now taken at a fixed height rather than full-page.

What was harness, not store

Of 136 failures in the last full live run, 129 were about the harness. Each has been fixed, and the fixes matter more than the count:

COUNT	FAILURE	CAUSE
30	"missing its {locale} copy"	Compared the live store against the fixture's catalogue. **Five were against English** — and English cannot be missing its own translation. The block was measuring the distance between two stores
68	HTTP 429	All on `/checkout`, while every other route answered 200. Shopify will not open a checkout with an empty cart

COUNT	FAILURE	CAUSE
27	"fits the mobile viewport"	Asserted a header selector that does not match this theme, as a proxy for "the page rendered"
4	`price "" does not match the EUR pattern`	The selector names the sale price, which is empty when a product is not discounted. A verdict on formatting, about a string nobody read

The English row in the first item is the point worth carrying out of this brief. It was a built-in control nobody had read, and it proved the entire block was measuring the wrong thing. Every check in this repository now has a control like it, and the release gate refuses to report a live result unless those controls are green.

6. Scheduled runs and cost

RUN	FREQUENCY	BLOCKING
Reconciliation — price, compare-at, inventory, status	Twice daily	Alerts on breach
Ad-traffic landing pages	Daily, ahead of the working day	Alerts
Top-10 sellers	Daily	Alerts
Translation parity — every translatable string, four locales	Nightly	No, trend-tracked
Translation backlog — the 93 open tasks	Nightly	No
Accessibility by market	Nightly	No
Full browser matrix	Nightly	No
Visual regression	Nightly	No, until the flake rate is known
Real-device smoke	Nightly	No
Pull-request gate — lint, types, smoke	Every pull request	Yes, under Option B
Launch gate	Before each launch	Yes

COST ITEM	PER MONTH
Framework, tooling, report hosting, digest	Nil — open source and existing accounts
CI compute, any platform	$0 to $40
Artifact storage under a retention policy	Under $5
Real-device grid — optional, nightly only, phase two	$79 to $199

Phase one: $0 to $45 a month. Phase two, with real devices: under $200. The material inputs are QA time, about two days of developer time, and a maintained staging store. The monthly bill is immaterial against any one of them.

7. What still needs to be built

Blocking a first complete live result

ITEM	EFFORT	BLOCKS
Map the cart selectors — `cart_line`, `cart_line_price`, `cart_line_remove`, `cart_subtotal`	30 minutes with someone who knows the theme	Welcome-kit parity in the cart
Map `site_header` on the live theme	Same session	Navigation and footer translation surfaces
Decide the bundle-builder approach — drive the builder, add by variant permalink, or declare it unrunnable	A decision, then up to a day	The whole welcome-kit flow live
Run the Shopify translations collector once and stamp the catalogue	An hour, once the token exists	Positive translation checks against the live store

The first two are the same 30 minutes. `KITSCH_BASE_URL=… npm run preflight` lists every selector that matched nothing; the list reaching empty is the definition of done.

Known coverage gaps — unverified, not passing

GAP	WHY	TO CLOSE
Live checkout, all locales	Not browsable with an empty cart	Seed a cart first — same blocker as the bundle builder
Positive copy on the live store	No live-store catalogue exists yet	Run the collector; point the baseline at its output
Korean and Japanese	The store does not serve them	Re-add to config if either market goes live

Engineering work

PRIORITY	ITEM	SIZE
P0	Config drift — `translation-gate.yml` and `config/a11y.yaml` still carry seven locales	Half a day
P0	Failure routing to Slack and the ticketing system	Two days
P1	Money and purchase path — bundles, cart, checkout, discount codes, subscription	~14 specifications
P1	Page-object and fixture libraries, before the specification count grows	Two days
P2	Discovery and rendering — collections, pagination, landing pages	~8 specifications
P3	Marketplace channel synchronisation, further application flows	~6 checks

A finding that is produced and never routed is worse than no finding, because it creates the appearance of coverage. That is why routing sits at P0 alongside the drift fix.

8. The credential question

The translation content layer needs a read-only Shopify Admin token, scoped `read_translations`, `read_products`, `read_online_store_pages` and nothing more. A lint rule fails the build if a GraphQL mutation ever appears in a collector.

Where that token lives is a decision, not a default. On a personal machine it sits outside device management, rotation and revocation. Held as a CI secret it never touches a laptop, and the same move makes the results reproducible. That is the stronger reason to enable the pipeline first and issue the token into it, rather than the other way round.

The same applies to the Asana token, which `asana:close` uses to close completed translation tasks.

---

9. Decisions requested

#	REQUEST	COST PER MONTH	BLOCKS
1	Merge `develop` into `main`, or repoint the default branch at `develop`, so scheduled workflows can fire. Name one owner	Nil	Every scheduled run. Nothing else in this brief matters until this is done
2	Issue a read-only Shopify Admin token and an Asana token, held as CI secrets rather than local files	$0 to $40 of compute	Translation content layer; automated task closure
3	Confirm a development or staging store with representative data, and who maintains it	Nil	All write-path coverage — the largest constraint on scope
4	Approve Option A: scheduled and launch tiers now, merge gate once the flake rate is known	Nil	Nothing. Can begin immediately
5	Name the recipient of the daily digest and the escalation route for critical findings	Nil	Whether findings result in fixes

Decision 1 is the whole of the pipeline ask. It is a branch operation of a few minutes, and it is worth being blunt about the current position: the automation has been proving itself correct on every push for a week while never once looking at the store on a schedule. That gap is not visible from a green badge, which is exactly why it went unnoticed.

Real devices are deliberately not on this list. They are worth putting forward once flake data exists, not before.

Timing and sequencing sit with the sponsor and QA lead. Nothing in this brief changes daily hands-on site testing, launch QA gates, or the framework build, all of which continue as they are. **None of the tooling described replaces manual testing.**

Appendix — supporting documents

All in the repository.

DOCUMENT	CONTENTS
`README`	Layers, repository layout, configuration, finding shape and severities, standing rules
`docs/LIVE-RUN.md`	The live runs: findings, harness problems, and the three named coverage gaps
`docs/ACCESS-REQUEST.md`	The credential and store asks, written to be forwarded
`docs/TEST-CASE-COVERAGE.md`, `docs/TRACEABILITY.md`	Risk areas, cited evidence, knowingly uncovered areas
`docs/ADDING-A-CHECK.md`	How a check is added, and why each needs a detection control
`docs/WELCOME-KIT-COVERAGE.md`	The bundle-builder decision, with the evidence behind each option

Cost figures are public list pricing retrieved on 1 September 2026 and should be confirmed against the organisation's actual plan before commitment.